LOW-LEVEL DESIGN (LLD)

RAG-Based Customer Support Assistant

Internship Project | 2026

1. Module-Level Design

1.1 Document Processing Module

File: load_pdf.py

Class Used: PyPDFLoader (LangChain)

Input: PDF file path (string)

Output: List of Document objects with page_content and metadata

Key Config: No mode parameter — default loads one Document per page

1.2 Chunking Module

File: chunking.py / graph.py

Class Used: RecursiveCharacterTextSplitter

chunk_size: 100 characters — keeps Q&A pairs together

chunk_overlap: 20 characters — preserves context between chunks

separators: ["\n\n", "\n", ". ", " ", ""] — tries paragraph break first

Input: List of Document objects

Output: List of smaller Document chunks with metadata preserved

1.3 Embedding Module

Model: all-MiniLM-L6-v2 (HuggingFace sentence-transformers)

Vector Dimensions: 384

Library: langchain-huggingface

Caching: Model downloaded once and cached locally by sentence-transformers

1.4 Vector Storage Module

Database: ChromaDB (in-memory / local)

Method: Chroma.from_documents(chunks, embeddings)

Storage: In-memory for this project; can be persisted with persist_directory

1.5 Retrieval Module

Method: vectorstore.as_retriever(search_kwargs={'k': 2})

k value: 2 — returns top 2 most semantically similar chunks

Search type: Cosine similarity on embedding vectors

1.6 Graph Execution Module (LangGraph)

Framework: LangGraph StateGraph

Nodes: retrieve_node, route_node, answer_node, hitl_node

Entry Point: retrieve_node

State Object: TypedDict with: question, context, answer, needs_human

1.7 HITL Module

Trigger: Context too short (<20 chars) OR complex keywords detected

Complex Keywords: legal, lawsuit, fraud, complaint, manager, refund denied

Action: Pauses graph, prompts human agent for input via CLI

Integration: Human response stored in state['answer'] and returned

2. Data Structures

2.1 Document Object (LangChain)

```
Document(
  page_content = 'Q: How do I return a product?...'  # extracted text
  metadata = {
    'source': 'custe.pdf',
    'page': 0,
    'total_pages': 1
  }
)
```

2.2 Graph State Object

```
class State(TypedDict):
  question:     str    # user's input query
  context:      str    # retrieved chunks joined as string
  answer:       str    # final answer (LLM or human)
  needs_human:  bool   # routing decision flag
```

2.3 Query-Response Schema

Input: {"question": "How do I return a product?", "context": "", "answer": "", "needs_human": False}

Output: {"question": "...", "context": "chunk text...", "answer": "You can return...", "needs_human": False}

3. LangGraph Workflow Design

3.1 Nodes

Node	Responsibility	State Input / Output
retrieve_node	Fetch relevant chunks from ChromaDB	IN: question | OUT: context
route_node	Check if escalation is needed	IN: context, question | OUT: needs_human
answer_node	Generate answer using LLM + context	IN: question, context | OUT: answer
hitl_node	Collect human agent response	IN: question | OUT: answer

3.2 Edges

retrieve --> route: Always runs after retrieval

route --> answer (conditional): When needs_human is False

route --> hitl (conditional): When needs_human is True

answer --> END: Graph terminates after answer

hitl --> END: Graph terminates after human response

4. Conditional Routing Logic

4.1 Escalation Criteria

```
def should_escalate(state):
  low_context = len(state['context'].strip()) < 20
  complex_keywords = ['legal','lawsuit','fraud','complaint','manager']
  is_complex = any(kw in state['question'].lower() for kw in
complex_keywords)
  return 'hitl' if (low_context or is_complex) else 'answer'
```

Low Context: If retrieved chunks have less than 20 characters, RAG cannot answer reliably — escalate

Complex Query: Legal, fraud, or complaint-type queries require human judgment — escalate

Normal Query: Sufficient context + simple intent — answer using LLM

5. HITL Design

5.1 When Escalation is Triggered

- Retrieved context is too short or empty (document does not contain relevant info)

- Question contains sensitive keywords such as legal, lawsuit, fraud, complaint
- In production: LLM confidence score below threshold (future enhancement)

5.2 What Happens After Escalation

- HITL node logs the question and notifies a human agent
- Human agent types their response (simulated via CLI in this project)
- Response is stored in state['answer'] and returned to the user

5.3 Production Integration

- In a real system: create a support ticket in Zendesk / Freshdesk via API
- Send email notification to on-call agent
- Poll for human response and resume graph execution

6. Interface Design

6.1 Input Format

```
result = app.invoke({
  "question": "How do I return a product?",
  "context": "",
  "answer": "",
  "needs_human": False
})
```

6.2 Output Format

```
result["answer"]  -->  "You can return any product within 30 days..."
```

7. Error Handling

Empty PDF: PyPDFLoader returns Document with empty page_content — system detects and skips

No Relevant Chunks: context will be empty string — route_node detects low_context and escalates to HITL

LLM Failure: Wrap llm.invoke() in try/except — return fallback message and log error

Invalid Query: Empty string check at entry point — return prompt asking for valid question